Especificação SDD TaskAnalyzer - Carlos Emanuel Santos da Silva

1. VISÃO GERAL E CONTRATO DE NEGÓCIO (SDD)

1.1 Identificação

Nome completo: Carlos Emanuel Santos da Silva
Curso: Análise e Desenvolvimento de Sistemas
Polo/Turma: Asa Norte, Turma A
E-mail institucional: carlos.emanuels@sempreceub.com

1.2 Propósito do Módulo TaskAnalyzer

O TaskAnalyzer tem como objetivo analisar um conjunto de tarefas e gerar métricas relacionadas à produtividade.

A partir dos dados recebidos, o módulo deverá calcular o tempo médio de conclusão das tarefas, a taxa de atraso e indicadores de produtividade separados por prioridade.

O módulo receberá os dados das tarefas, realizará as validações necessárias, processará as informações conforme esta especificação e retornará os resultados de maneira organizada e consistente.

1.3 Contrato Executável de Interface

Entrada

O TaskAnalyzer deverá receber uma lista de tarefas. Cada tarefa deverá seguir a estrutura abaixo.

Campo	Tipo	Descrição
`id`	inteiro	Identificador único
`titulo`	texto	Título da tarefa
`prioridade`	texto	Prioridade da tarefa
`data_criacao`	data/hora	Data e hora de criação
`data_limite`	data/hora	Prazo para conclusão
`data_conclusao`	data/hora ou nulo	Data e hora de conclusão
`status`	texto	Situação atual da tarefa

Prioridades aceitas: BAIXA, MEDIA e ALTA.

O campo `data_conclusao` poderá possuir valor nulo enquanto a tarefa ainda não estiver concluída.

Saída

Após o processamento, o TaskAnalyzer deverá retornar:

Métrica	Descrição
Tempo médio de conclusão	Média do tempo entre criação e conclusão das tarefas concluídas
Taxa de atraso	Percentual de tarefas concluídas após a data limite
Indicadores por prioridade	Métricas agrupadas por BAIXA, MEDIA e ALTA

Os resultados deverão ser retornados sempre em uma estrutura consistente.

Regras e validações

1. A entrada deverá ser uma lista de tarefas válida.
2. Os dados deverão ser validados antes dos cálculos.
3. Cada tarefa deverá possuir um `id` único.
4. A prioridade deverá ser BAIXA, MEDIA ou ALTA.
5. Os campos de data deverão possuir valores válidos.
6. Dados inválidos não poderão ser utilizados no cálculo das métricas.
7. Cálculos dependentes de conclusão utilizarão somente tarefas que possuam `data_conclusao`.
8. Uma lista vazia deverá ser tratada de forma previsível.
9. Entradas inválidas deverão gerar o código de erro correspondente.
10. A estrutura de saída deverá permanecer consistente.

Tratamento de erros

Código	Situação
`INVALID_TASK_DATA`	Dados da tarefa não seguem a estrutura definida
`INVALID_PRIORITY`	Prioridade não permitida
`INVALID_DATE`	Data informada é inválida
`DUPLICATE_TASK_ID`	Identificador duplicado
`EMPTY_TASK_LIST`	Lista de tarefas vazia

2. CENÁRIOS DE ACEITE E TEST HARNESS

Os cenários de aceite estabelecem antecipadamente o comportamento esperado do TaskAnalyzer e servirão como referência para os testes da etapa de implementação.

Cenário 1: Tempo médio de conclusão

Dado um conjunto válido de tarefas concluídas com datas de criação e conclusão,
Quando o TaskAnalyzer processar as tarefas,
Então deverá calcular e retornar o tempo médio de conclusão.

Cenário 2: Taxa de atraso

Dado um conjunto de tarefas concluídas com datas limite e datas de conclusão,
Quando o TaskAnalyzer analisar as tarefas,
Então deverá identificar as tarefas concluídas fora do prazo e calcular a taxa de atraso.

Cenário 3: Indicadores por prioridade

Dado um conjunto de tarefas classificadas como BAIXA, MEDIA ou ALTA,
Quando o TaskAnalyzer processar os dados,
Então deverá apresentar os indicadores correspondentes a cada prioridade.

Cenário 4: Lista vazia

Dado que nenhuma tarefa tenha sido fornecida,
Quando uma análise for solicitada,
Então deverá ser retornado o erro `EMPTY_TASK_LIST`.

Cenário 5: Dados inválidos

Dado uma tarefa que não siga a estrutura definida,
Quando o TaskAnalyzer tentar processá-la,
Então deverá ser retornado o erro `INVALID_TASK_DATA`.

Cenário 6: Prioridade inválida

Dado uma tarefa cuja prioridade não seja BAIXA, MEDIA ou ALTA,
Quando os dados forem validados,
Então deverá ser retornado o erro `INVALID_PRIORITY`.

Cenário 7: Identificador duplicado

Dado duas tarefas com o mesmo identificador,
Quando a lista for validada,
Então deverá ser retornado o erro `DUPLICATE_TASK_ID`.

Cenário 8: Data inválida

Dado uma tarefa contendo uma data inválida,
Quando os dados forem validados,
Então deverá ser retornado o erro `INVALID_DATE`.

2.1 Planejamento do Test Harness

Na etapa de implementação, os cenários de aceite serão utilizados como base para criação dos testes automatizados.

Os testes deverão verificar entradas válidas e inválidas, cálculo das métricas, prioridades, datas, identificadores duplicados, situações de limite, tratamento de erros e consistência dos resultados.

A implementação será considerada compatível com esta especificação quando o comportamento obtido nos testes estiver de acordo com os cenários de aceite definidos.

3. GOVERNANÇA DE CONTEXTO E REGRAS PARA AGENTES DE IA

O arquivo `CONTEXT_RULES.md` será utilizado durante a implementação para estabelecer as regras que deverão ser respeitadas pelos agentes de inteligência artificial.

O objetivo é manter o desenvolvimento alinhado à especificação, evitar alterações não planejadas e garantir que o código produzido possa ser revisado e testado.

3.1 Diretrizes de arquitetura e desenvolvimento

Durante a implementação deverão ser seguidas as seguintes diretrizes:

- utilizar Python 3.11 ou superior;
- utilizar type hints;
- seguir as recomendações da PEP 8;
- utilizar docstrings no padrão Google;
- utilizar nomes claros e descritivos;
- criar funções pequenas e com responsabilidades bem definidas;
- manter a lógica de negócio separada das operações de entrada e saída;
- produzir mensagens de exceção específicas;
- utilizar logging estruturado quando necessário;
- manter o código legível, organizado e testável;
- evitar dependências externas que não tenham sido autorizadas.

3.2 Regras para utilização de IA

Durante o desenvolvimento, a IA deverá consultar a especificação antes de realizar alterações, respeitar os contratos de entrada e saída, preservar os códigos de erro, implementar somente as funcionalidades previstas, respeitar os cenários de aceite, manter as assinaturas públicas definidas, produzir ou atualizar testes quando necessário e informar sempre que uma solicitação entrar em conflito com a especificação.

3.3 Restrições para agentes de IA

A IA não deverá criar requisitos sem autorização, assumir comportamentos não especificados, alterar ou remover cenários de aceite para facilitar a implementação, modificar testes para esconder erros, ignorar falhas encontradas, adicionar bibliotecas externas sem autorização, alterar a estrutura prevista do projeto, alterar assinaturas públicas sem autorização, persistir os dados em arquivos ou banco de dados, gerar código sem os testes correspondentes ou produzir código duplicado e desnecessário.

Qualquer mudança que altere o comportamento esperado do TaskAnalyzer deverá ser refletida primeiro nesta especificação.